SignLang: An AI-Driven Interactive Tutoring System with Gesture Recognition for Pakistan Sign Language (PSL)

Project Team

Syed Saad kabeer	22P-9032
Sana Ullah	22P-9038

Session 2022–2026

Supervised by

Dr. Muhammad Amin

Department of Computer Science
National University of Computer and Emerging Sciences
Peshawar, Pakistan

October, 2025

Student's Declaration

We hereby declare that the thesis titled *"SignLang: An AI-Driven Interactive Tutoring System with Gesture Recognition for Pakistan Sign Language (PSL)"*, submitted is our own, original work, completed independently without any unauthorized assistance. Only the sources and resources explicitly cited have been utilized. All excerpts, whether quoted directly or paraphrased, have been properly acknowledged. If generative AI tools were used, we have clearly indicated the product name, manufacturer, software version, and the specific purpose (e.g., language enhancement, systematic research). We take full responsibility for the selection, application, and interpretation of any AI-generated content incorporated into this work. A list of prompts along with corresponding page numbers where they are used is included in the appendix".

Student 1:	Syed Saad Kabeer	**Signature:** _____
Student 2:	Sana Ullah	**Signature:** _____

_____

Verified by Plagiarism Cell Officer

Dated: _____

Certificate of Approval

The Department of Computer Science, National University of Computer and Emerging Sciences, accepts this thesis titled *SignLang: An AI-Driven Interactive Tutoring System with Gesture Recognition for Pakistan Sign Language (PSL)*, submitted by Syed Saad Kabeer (22P-9032), SanaUllah (22P-9038), in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor
Dr. Muhammad Amin Signature: _____

FYP Coordinator Signature: _____

Head of Department (Computer Science) Signature: _____

National University of Computer and Emerging Sciences, Peshawar

Acknowledgements

We would like to express our heartfelt gratitude to our supervisor, Dr. Muhammad Amin, for his invaluable guidance, continuous support, and insightful feedback throughout the development of this project.

We extend our sincere thanks to the Special Complex Peshawar for their cooperation and assistance in enabling us to work on Pakistan Sign Language (PSL), and for providing valuable insights that helped us design and standardize the PSL content for our web-based learning and evaluation platform.

We are also grateful to the National University of Computer and Emerging Sciences (FAST-NUCES) for providing the necessary resources, technical facilities, and a productive academic environment that contributed to the successful completion of our project, SignLang: An AI-Powered Interactive Tutoring System Using Pakistan Sign Language (PSL).

Finally, we thank our families and friends for their constant encouragement, patience, and motivation throughout this journey.

Syed Saad Kabeer
Sana Ullah

Abstract

SignLang is an AI-driven interactive tutoring system designed to facilitate the learning of Pakistan Sign Language (PSL) through gesture recognition and real-time feedback. The system integrates Computer Vision and Deep Learning models to detect and evaluate hand gestures using live camera input, ensuring accurate PSL posture recognition. Recognized signs are mapped to English and Urdu text, enhancing both comprehension and communication. The web-based platform enables students and tutors to engage through interactive lessons, quizzes, and AI-assisted evaluations. By promoting standardized PSL learning and inclusivity for the deaf and hard-of-hearing community, SignLang aligns with UN SDGs 4, 9, and 10, advancing equitable education and innovation in assistive technology.

Contents

Student's Declaration **1**

Certificate of Approval **2**

Acknowledgements **3**

Abstract **4**

1 Introduction **10**

1.1 Overview 10
1.2 Motivation 10
1.3 Problem Statement 11
1.4 Project Scope 11
1.5 Objectives 11
1.6 Expected Outcomes 12
1.7 SDG Alignment 12

2 Review of Literature **13**

2.1 Introduction 13
2.2 Pakistan Sign Language Recognition 13
2.3 Deep Learning Approaches in PSL 13
2.4 Challenges and Research Gaps 14
2.5 Literature Review Summary 14

3 Project Vision **15**

3.1 Problem Statement (Revisited) 15
3.2 Business Opportunity 15
3.3 Goals and Objectives 15
3.4 Constraints 16
3.5 Expected Impact 16

4 Software Requirements Specifications **17**
4.1 Overview . . . 17
4.2 List of Features . . . 17
4.3 Functional Requirements . . . 17
4.4 Non-Functional Requirements . . . 18
4.5 Use Cases . . . 18
4.6 User Interaction Flow . . . 18
4.6.1 User Login and Authentication . . . 18
4.6.2 Query Submission and Analysis . . . 18
4.7 Response Generation . . . 19
4.8 Activity Sequence . . . 19

5 Iteration Plan **20**
5.1 Overview of Iterations . . . 20
5.2 System Architecture Overview . . . 20
5.3 Query Flow Representation . . . 20
5.4 Data Flow in the System . . . 21

6 Iteration 1 **22**
6.1 Objectives Achieved . . . 22
6.2 Structural Design . . . 22
6.2.1 System Architecture . . . 22
6.2.2 Use Case Diagram . . . 23
6.3 Behavioral Design . . . 24
6.3.1 Sequence Diagram . . . 24
6.3.2 Activity Diagram . . . 25
6.4 Data Flow Representation . . . 26
6.5 Iteration Summary . . . 27

7 Iteration 2 **28**
7.1 Overview . . . 28
7.2 Objectives Achieved . . . 28
7.3 Updated System Design . . . 28
7.3.1 Updated Architecture Diagram . . . 29
7.3.2 Updated Sequence Diagram . . . 29
7.4 AI Model Improvement . . . 30
7.5 Dataset Expansion . . . 30
7.6 Frontend Technology Finalization (Prototype Phase) . . . 31

7.7 Iteration Summary 31

8 Iteration 3 **32**

8.1 Overview 32
8.2 Objectives Achieved 32
8.3 Technology Stack Finalization 33
8.4 Updated User Interface 33
8.5 New Features Introduced 33
8.5.1 Cognitive Analysis Questionnaire 33
8.5.2 Neuro-Game 34
8.5.3 Lessons and Evaluation Module 35
8.5.4 Video Lectures 35
8.6 Backend Implementation with Node.js, Next.js API Routes, and TypeScript 36
8.7 System Integration and Testing 36
8.8 Iteration Summary 37

9 Implementation Details **38**

9.1 Overview 38
9.2 Technology Stack 38
9.2.1 Frontend 38
9.2.2 Backend 38
9.2.3 AI Model 39
9.2.4 Database and Authentication 39
9.2.5 State Management and Data Fetching 39
9.2.6 Form Handling and Validation 39
9.2.7 Version Control and DevOps 39
9.2.8 Clone Command 40
9.3 Environment Setup 40
9.3.1 Prerequisites 40
9.3.2 Verify Installation 40
9.4 Build Commands 40
9.4.1 Install Dependencies 40
9.4.2 Environment Configuration 40
9.4.3 Database Setup 41
9.4.4 Development Server 41
9.4.5 Production Build 41
9.5 AI Model Setup 41
9.5.1 Python Environment 41

9.5.2 Run Recognition Server 42
9.6 Complete Development Workflow 42
9.7 Production Deployment 42
9.7.1 Vercel Deployment 42
9.7.2 Recommended Production Steps 43
9.8 Chapter Summary 43

10 User Manual **44**
10.1 Overview 44
10.2 System Access 44
10.2.1 Web Application Access 44
10.3 Account Registration and Login 44
10.3.1 Creating a New Account 44
10.3.2 Logging In 45
10.3.3 Password Recovery 45
10.4 Student User Guide 45
10.4.1 Dashboard Navigation 45
10.4.2 Taking Lessons 45
10.4.3 Attempting Quizzes 46
10.4.4 Tracking Progress 46
10.5 Tutor User Guide 47
10.5.1 Course and Lesson Management 47
10.5.2 Quiz Creation 47
10.5.3 Student Performance Monitoring 47
10.6 Gesture Recognition Interaction 48
10.6.1 Camera Setup 48
10.6.2 Performing Signs 48
10.7 Troubleshooting 49
10.8 Accessibility Features 49

11 Conclusion and Future Work **50**
11.1 Conclusion 50
11.2 Future Work 51
11.2.1 Expansion of Content Library 51
11.2.2 Integration with Official PSL Website 51
11.2.3 AI-Based Adaptive Learning 52
11.2.4 Enhanced Recognition Capabilities 52
11.2.5 Community and Social Features 52

11.2.6 Research and Dataset Contributions 53
11.3 Final Remarks 53

Bibliography **54**

Appendix A: AI Usage Details **55**

Appendix B: External Tools and Technologies Used **57**

Chapter 1

Introduction

1.1 Overview

SignLang is an AI-powered interactive learning platform that enables users to learn Pakistan Sign Language (PSL) through real-time gesture recognition and feedback. The system employs Computer Vision and Deep Learning techniques to identify and evaluate PSL gestures captured via a camera, ensuring accuracy and adherence to standardized signs. It offers a web-based interface where students can take lessons, practice signs, and complete AI-assisted evaluations, while tutors can create customized lessons, track progress, and analyze performance trends. By combining AI-driven learning, interactive assessments, and bilingual support (English, & Urdu), SignLang aims to make PSL education more accessible, inclusive, and standardized for the deaf and hard-of-hearing community in Pakistan.

1.2 Motivation

In Pakistan, the **deaf and hard-of-hearing community** faces significant barriers in accessing quality education due to the limited availability of trained sign language instructors and standardized learning resources. Existing teaching methods for **Pakistan Sign Language (PSL)** are largely manual, lacking interactivity, accessibility, and real-time feedback. This gap creates challenges in effective communication and social inclusion for hearing-impaired individuals.

The motivation behind **SignLang** is to harness the power of **Artificial Intelligence (AI)** and **Computer Vision (CV)** to make PSL learning **interactive, standardized, and accessible** for everyone. By enabling real-time gesture recognition, evaluation, and guided learning through a web-based platform, the project aims to **empower learners and educators**, promote **inclusive education**, and bridge the communication divide between hearing and non-hearing communities in Pakistan.

1.3 Problem Statement

In Pakistan, the lack of standardized and accessible platforms for learning **Pakistan Sign Language (PSL)** limits educational opportunities for the deaf and hard-of-hearing community. Traditional sign language teaching methods rely heavily on manual instruction, offering minimal interactivity and no real-time feedback. This creates inconsistencies in learning accuracy and slows the adoption of PSL across institutions. Furthermore, there is limited technological integration to assess and evaluate learners' gestures effectively. Therefore, there is a need for an **AI-driven interactive tutoring system** that can recognize, evaluate, and guide PSL gestures in real time, promoting **standardized, inclusive, and efficient learning**.

1.4 Project Scope

1. Data collection and preprocessing of Pakistan Sign Language (PSL) gesture datasets.
2. Training and fine-tuning of AI models for hand gesture recognition and pose estimation.
3. Mapping recognized PSL gestures to corresponding English and Urdu text.
4. Development of an interactive web-based learning platform for students and tutors.
5. Integration of real-time feedback and performance evaluation using AI-based accuracy analysis.
6. Implementation of tutor tools for lesson creation, quiz management, and progress tracking.
7. Deployment of the system for accessibility across desktop, mobile, and Smart TV platforms.

1.5 Objectives

- Develop an AI-driven tutoring system capable of recognizing and evaluating Pakistan Sign Language (PSL) gestures in real time.
- Convert recognized gestures into corresponding English and Urdu text for better understanding and accessibility.

- Provide interactive lessons, quizzes, and AI-based feedback to enhance learning accuracy and engagement.
- Enable tutors to create, manage, and track lessons and student performance using intelligent analytics.
- Deploy the system as a responsive web-based platform accessible via desktop, mobile, and Smart TV.

1.6 Expected Outcomes

- A trained AI model for accurate PSL gesture recognition using computer vision and deep learning.
- A fully functional web-based learning platform supporting interactive PSL lessons and real-time evaluations.
- Tools for tutors to create content, manage quizzes, and monitor student progress through AI-generated insights.
- Bilingual support (English and Urdu) for improved communication and inclusivity.
- An accessible and standardized platform promoting PSL education nationwide.

1.7 SDG Alignment

- **Goal 4:** Quality Education – Ensure inclusive and equitable education for the deaf and hard-of-hearing community.
- **Goal 9:** Industry, Innovation, and Infrastructure – Promote innovation in assistive and educational technologies using AI.
- **Goal 10:** Reduced Inequalities – Empower persons with disabilities by providing equal access to digital learning tools.

Chapter 2

Review of Literature

2.1 Introduction

Sign Language Recognition (SLR) has gained significant attention in recent years as a crucial step toward bridging communication gaps between the hearing and speech-impaired communities and the rest of society. With the advancements in computer vision and deep learning, researchers have focused on recognizing hand gestures and postures to automate the interpretation of sign languages, including Pakistan Sign Language (PSL). However, challenges remain due to limited datasets, variations in lighting and background, and the complexity of continuous sign gestures.

2.2 Pakistan Sign Language Recognition

Several studies have explored PSL recognition using machine learning and deep learning techniques. Imran et al. (2021) developed a dataset for PSL and focused on recognizing hand configurations of Urdu alphabets using traditional machine learning methods. Their work laid the foundation for PSL dataset availability. Mirza et al. (2022) further enhanced recognition accuracy using a Bag-of-Words model and Support Vector Machines (SVM), proving that vision-based models can handle PSL with reasonable success.

2.3 Deep Learning Approaches in PSL

Recent work has shifted toward deep learning due to its superior performance in image and gesture recognition tasks. Hamza and Wali (2023) demonstrated the use of deep learning models for PSL recognition even with limited data, emphasizing transfer learning as a viable approach. Similarly, Arooj et al. (2024) combined Convolutional Neural Networks (CNN) with Scale-Invariant Feature Transform (SIFT) to improve feature extraction and classification accuracy in PSL datasets.

2.4 Challenges and Research Gaps

While significant progress has been made, several challenges persist. Zahid et al. (2022) conducted a systematic review highlighting issues such as inconsistent datasets, lack of standardization, and the need for real-time recognition systems. Moreover, Khan et al. (2021) discussed the broader challenges in developing PSL through information technology, including limited community engagement, insufficient linguistic resources, and lack of government-backed standardization.

2.5 Literature Review Summary

No.	Literature	Year	Methodology	Limitations
1	Imran et al.	2021	Created PSL dataset; used ML for Urdu alphabet recognition	Limited to static hand shapes
2	Mirza et al.	2022	Vision-based PSL recognition using BoW + SVM	Dependent on image quality
3	Zahid et al.	2022	Systematic review of ML classifiers for Urdu SL	Lacks implementation and real-time testing
4	Hamza & Wali	2023	Deep learning with limited PSL data using transfer learning	Dataset too small for robust models
5	Arooj et al.	2024	CNN + SIFT hybrid approach for PSL gesture recognition	Focused on static gestures only
6	Khan et al.	2021	Discussion on developing PSL using IT	Conceptual; lacks technical implementation

Chapter 3

Project Vision

3.1 Problem Statement (Revisited)

In Pakistan, the hearing-impaired community faces significant barriers to education and communication due to the limited availability of standardized and accessible learning tools for Pakistan Sign Language (PSL). Existing methods rely on manual instruction or non-interactive video tutorials, which fail to provide personalized feedback or real-time gesture evaluation. Moreover, PSL lacks a unified digital learning platform that aligns with officially recognized standards and is easily deployable across classrooms, Smart TVs, and personal devices.

3.2 Business Opportunity

- **Educational Institutions** — Schools and special education centers can integrate the system into their curriculum to promote inclusive learning.
- **Tutors and Trainers** — Instructors can create customized PSL lessons, track student progress, and receive AI-driven insights on gesture performance.
- **Students and Learners** — Individuals can practice and learn PSL interactively through real-time gesture recognition and accuracy feedback.
- **Organizations and NGOs** — Assistive technology programs can adopt this solution to train staff or provide accessible communication tools.

3.3 Goals and Objectives

The primary goal of this project is to design and develop an AI-driven, web-based tutoring system that enables real-time learning and evaluation of Pakistan Sign Language. The

objectives include:

- Implement computer vision models for gesture recognition and accuracy evaluation.
- Convert recognized signs into English and Urdu text for accessibility.
- Develop interactive modules with quizzes and assignments for student assessment.
- Enable tutors to create, manage, and evaluate lessons via a centralized portal.
- Deploy the platform on web and Smart TV interfaces to enhance accessibility.

3.4 Constraints

- **Dataset Limitations** — Lack of large-scale, standardized PSL datasets for model training.
- **Model Complexity** — Real-time gesture detection requires efficient yet accurate deep learning models.
- **Hardware Dependence** — Performance varies based on camera quality and system processing power.
- **Standardization** — PSL still lacks complete official standardization, making linguistic alignment challenging.
- **Deployment and Maintenance** — Ensuring consistent performance across multiple devices and platforms.

3.5 Expected Impact

The proposed system will have a transformative effect on the accessibility and quality of PSL education in Pakistan. It will:

- Standardize PSL learning and teaching through AI-assisted evaluation.
- Enhance inclusivity by providing equal learning opportunities to deaf and hard-of-hearing individuals.
- Empower teachers through data-driven insights on student progress.
- Encourage adoption of assistive AI technologies in the education sector.
- Contribute to the Sustainable Development Goals (SDG 4, SDG 9, and SDG 10) by promoting quality education, innovation, and reduced inequalities.

Chapter 4

Software Requirements Specifications

4.1 Overview

This chapter outlines the functional and non-functional requirements of the SignLang tutoring system. It defines the system features, use cases, user interaction flow, and activity diagrams that form the foundation for development and evaluation.

4.2 List of Features

- Real-time PSL gesture recognition using AI model.
- Interactive lesson modules and quizzes.
- Student portal for lessons, evaluation, and progress tracking.
- User authentication.
- Web-based deployment.

4.3 Functional Requirements

1. The system shall authenticate all users.
2. The system shall detect and classify PSL gestures using a camera.
3. The system shall evaluate gesture accuracy and provide real-time feedback.
4. The system shall track user progress for lessons and tests.
5. The system shall store lesson content, evaluation data, and profiles.

4.4 Non-Functional Requirements

- **Performance:** Gesture recognition must produce results within 0.5–1.5 seconds.
- **Usability:** The user interface must be responsive and accessible.
- **Scalability:** Capable of supporting multiple simultaneous users.
- **Security:** Passwords encrypted; role-based access enforced.
- **Reliability:** System uptime must exceed 95%.
- **Compatibility:** Works on desktop, mobile, and Smart TV browsers.

4.5 Use Cases

- **Use Case 1: User Login** – User logs in with email/password.
- **Use Case 2: Lessons with PSL Signs** – Student practices correct signs of PSL language, related to their course content.
- **Use Case 3: Attempt Quiz** – Student attempts PSL accuracy-based quiz, to verify their knowledge for the course material as well as correct PSL gestures.
- **Use Case 4: Track Progress** – System displays performance analytics.

4.6 User Interaction Flow

4.6.1 User Login and Authentication

1. User enters credentials.
2. System verifies user identity.
3. User is redirected to student/tutor dashboard based on role.

4.6.2 Query Submission and Analysis

1. User starts sign test.
2. Camera captures hand gesture in accordance to the word.
3. AI model processes gesture and predicts PSL sign.

4. UI displays recognized sign and accuracy.
5. If the predicted Sign is correct, user moves onto another word or question.

4.7 Response Generation

- AI model returns predicted label.
- System calculates accuracy percentage.
- Feedback is displayed visually and textually.

4.8 Activity Sequence

- User logs in.
- Selects lessons from the dashboard.
- Starts course lessons with PSL gesture recognition.
- AI model detects hand pose in accordance to words.
- Accuracy and feedback shown on screen for the selected word.

Chapter 5

Iteration Plan

5.1 Overview of Iterations

The development of SignLang follows an incremental and iterative model. Each iteration delivers functional components, allowing evaluation, testing, and refinement.

- **Iteration 1:** System design, architecture, diagrams, basic UI layout.
- **Iteration 2:** Prototype of gesture recognition model and Web UI.
- **Iteration 3:** Finalization of Web Interface and full lesson and quiz module, along with complete AI model.
- **Iteration 4:** Analytics, optimization, deployment.

5.2 System Architecture Overview

The system adopts a modular architecture consisting of:

- Frontend (Next.js 14+ web interface with React 18+ and TypeScript)
- Backend (Node.js with Next.js API Routes, tRPC, and Prisma ORM)
- AI Model Server (Python with FastAPI, TensorFlow/Keras, MediaPipe, and OpenCV)
- Database (Supabase with PostgreSQL)

5.3 Query Flow Representation

1. User performs a PSL gesture.
2. Camera feed is streamed to AI Recognition API (Python/FastAPI server).

3. AI model returns predicted sign.
4. User interface displays result and confidence score.

5.4 Data Flow in the System

1. User data stored in database (Supabase).
2. Lesson and quiz data delivered to frontend on demand via Next.js API routes.
3. Tutor dashboards retrieve all lessons made for students through tRPC/Prisma queries.

Chapter 6

Iteration 1

6.1 Objectives Achieved

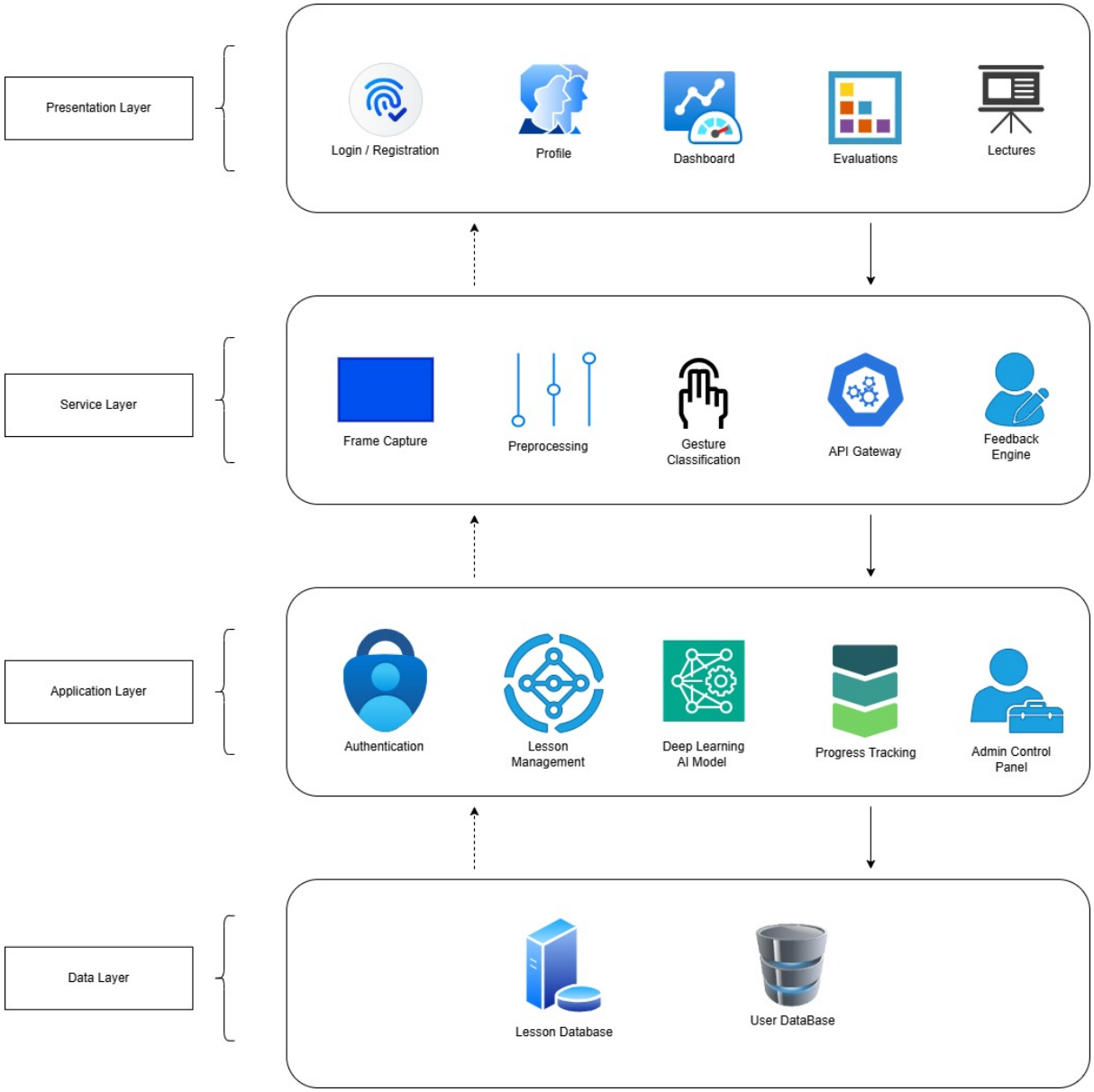
- Defined system architecture.
- Created prototype Web UI (Next.js-based).
- Created user login / registration.
- Made a prototype user dashboard with Test, Lessons and Profile options.
- Built prototype AI model for PSL character recognition.
- Developed initial architectural diagrams.

6.2 Structural Design

6.2.1 System Architecture

The system layered architecture illustrates interactions between:

- Presentation Layer: Web Interface (Next.js Front-end)
- Service Layer: Backend Server / API Layer (Node.js/Next.js API Routes)
- Application Layer: AI Model Module (Python/FastAPI)
- Data Layer: Database Management System (Supabase)

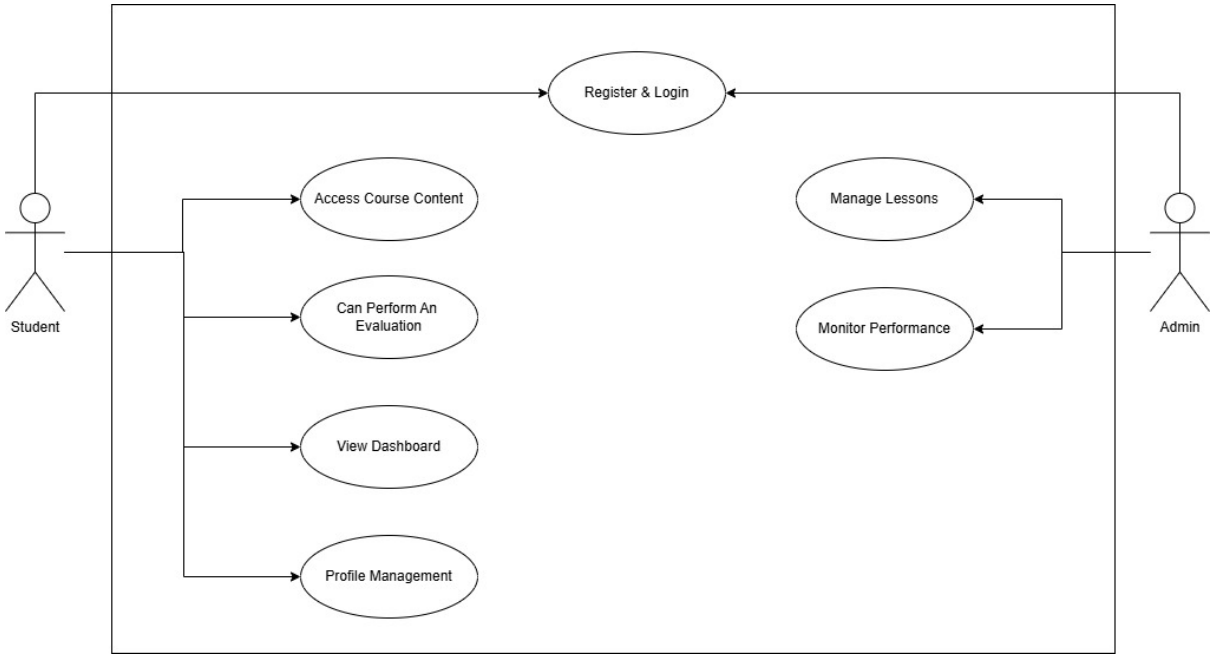


6.2.2 Use Case Diagram

The primary actors are:

- User (Student + Tutor)
- Admin

Use cases include login, gesture recognition, quiz interaction, lesson creation, and progress monitoring.

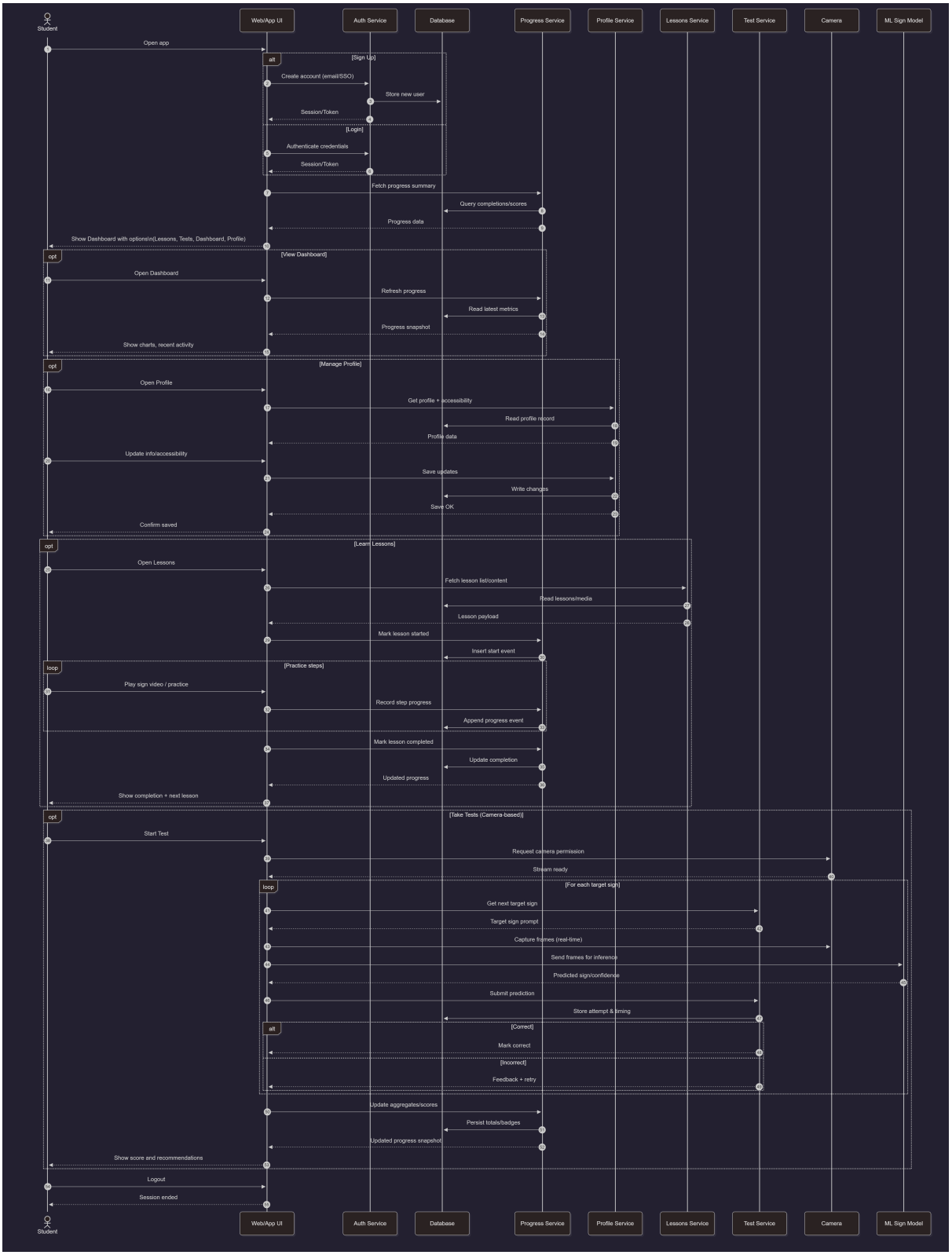


6.3 Behavioral Design

6.3.1 Sequence Diagram

The sequence consists of:

1. User requests gesture evaluation.
2. Camera captures input.
3. AI model predicts gesture.
4. System sends back recognized sign + accuracy.



6.3.2 Activity Diagram

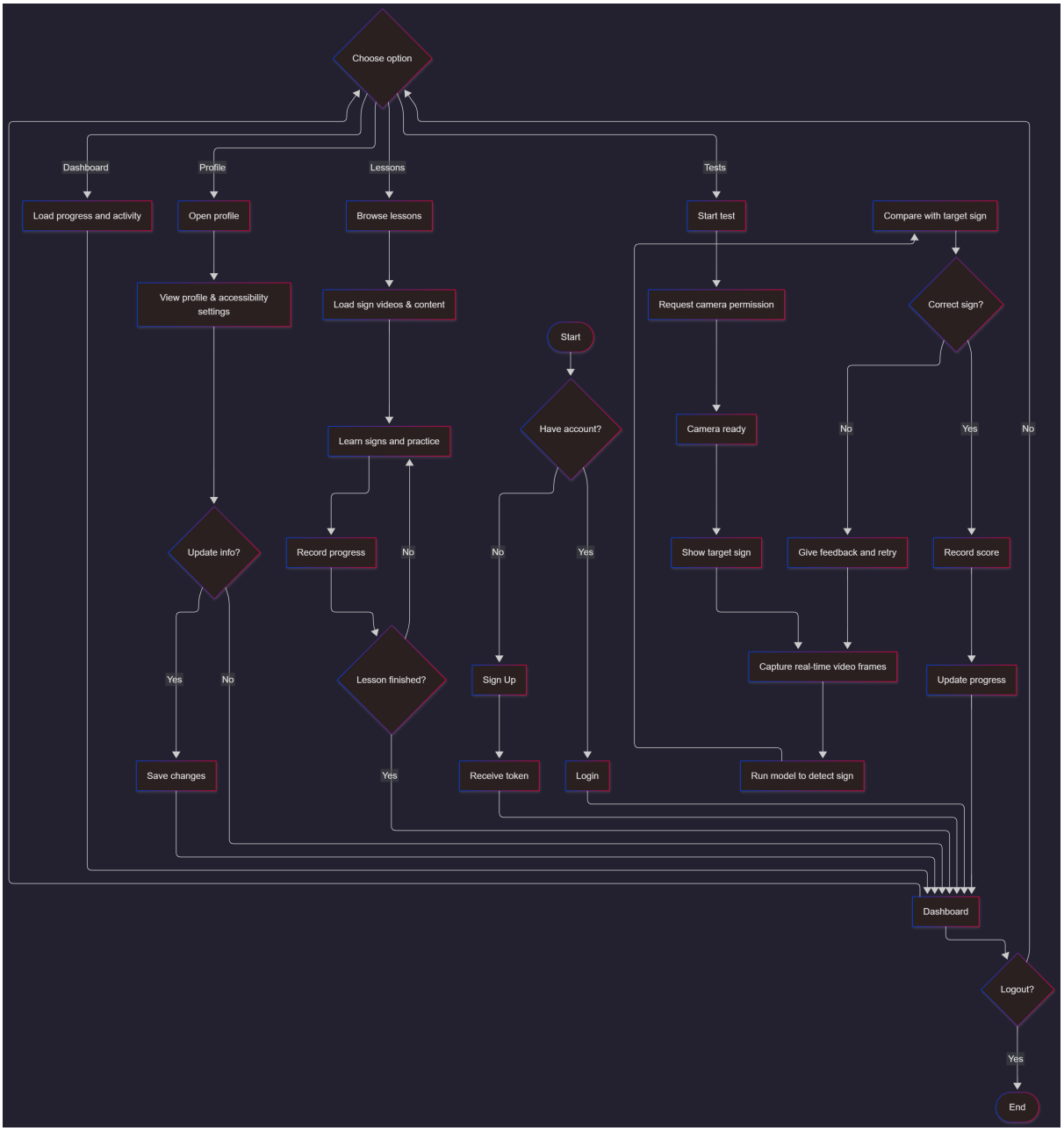
The activity flow includes:

1. Start recognition.

2. Capture frames.

3. Process gesture.

4. Evaluate and display results.



6.4 Data Flow Representation

Iteration 1 includes preliminary data flow diagrams showing the movement of gesture data, system requests, and analytics.

6.5 Iteration Summary

Iteration 1 established the core foundation for system development, including architecture, requirement gathering, structural design, and workflow modeling. These components provide a stable groundwork for implementing gesture recognition and interactive learning modules in future iterations.

Chapter 7

Iteration 2

7.1 Overview

Iteration 2 focused on refining the initial prototype into a more robust and practical version of the SignLang system. In this phase, the previously designed modules were improved with updated diagrams, a more mature gesture recognition pipeline, and a clearer implementation strategy for the final system. The objective of this iteration was to move beyond the conceptual and prototype stage and develop a stronger technical foundation for real-world deployment.

7.2 Objectives Achieved

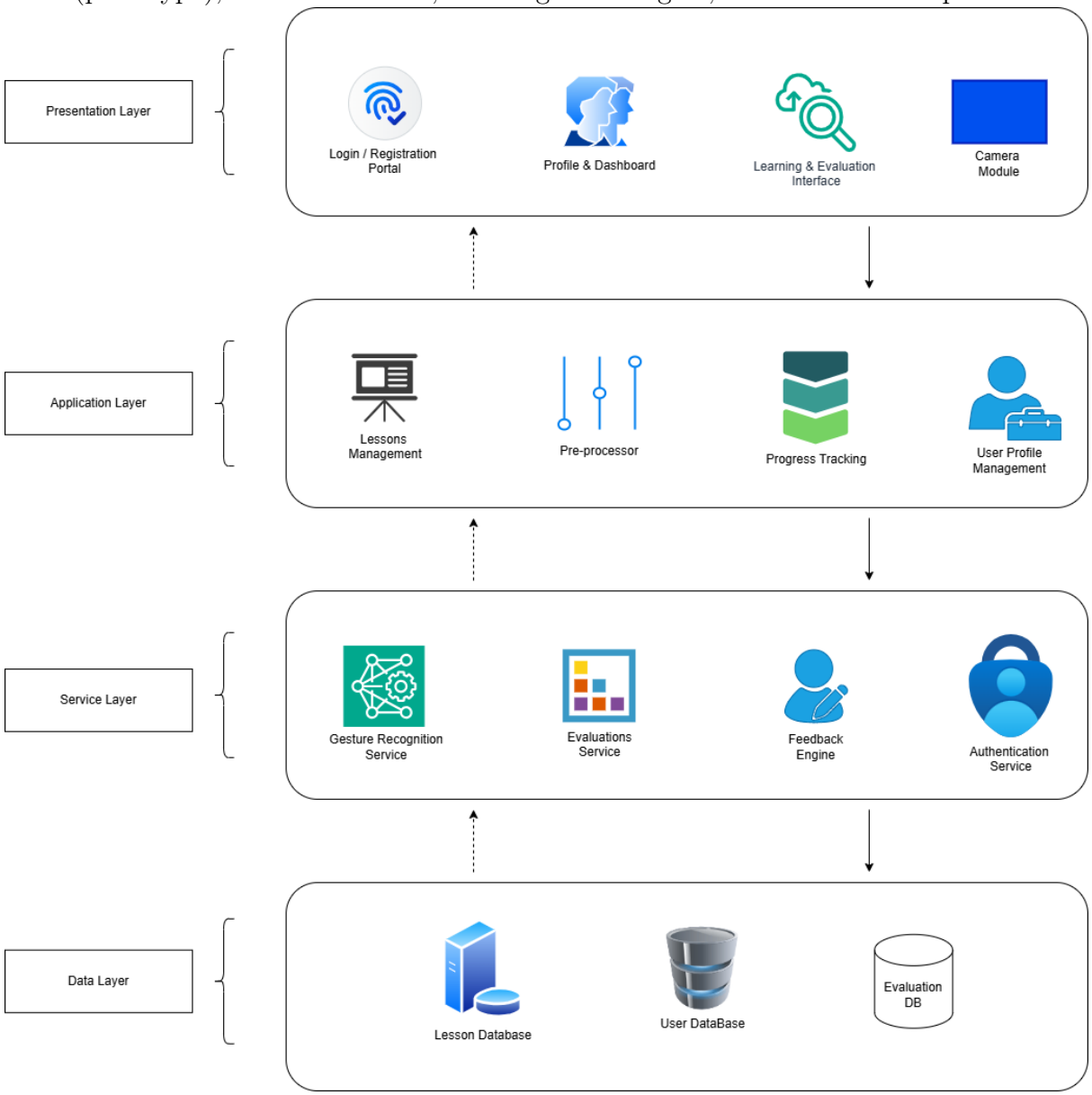
- Updated and improved the system diagrams developed in Iteration 1.
- Refined the workflow of user interaction and AI-based gesture evaluation.
- Strengthened the AI model training pipeline using Google InceptionV3.
- Expanded the dataset used for training to approximately 40,000 images.
- Finalized the decision to use Flutter for the application interface (prototype phase).
- Enhanced system design for better scalability, responsiveness, and maintainability.

7.3 Updated System Design

During this iteration, the structural and behavioral design artifacts of the system were revised to better represent the actual implementation flow. Based on feedback and further development, the diagrams were updated to reflect clearer communication between the user, application, camera module, and AI recognition model.

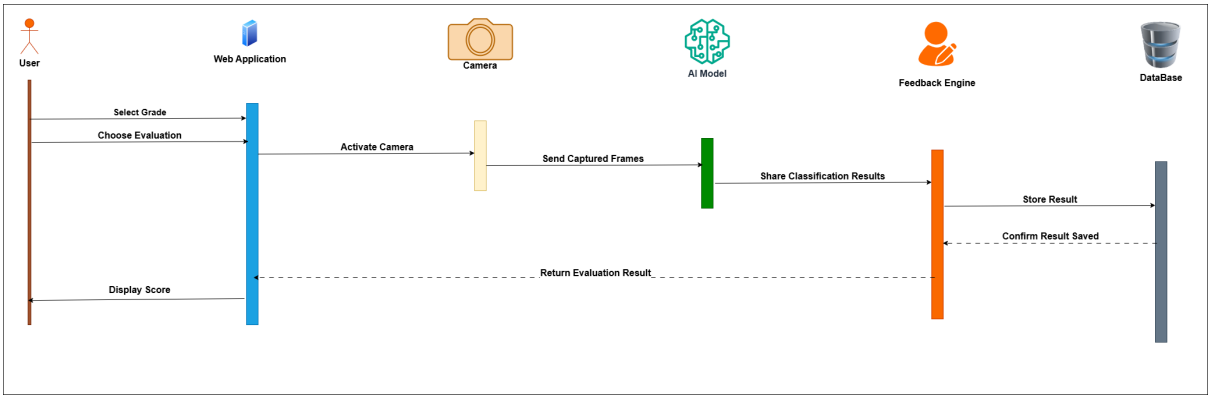
7.3.1 Updated Architecture Diagram

The architecture diagram was revised to represent the finalized modules of the Sign-Lang system more accurately. It now reflects the interaction between the Flutter-based frontend (prototype), backend services, AI recognition engine, and database components.



7.3.2 Updated Sequence Diagram

The sequence diagram was refined to show the correct sequence of events during gesture evaluation. It now presents a clearer flow in which the user initiates an evaluation task, the application activates the camera, the captured frames are processed by the AI model, and the result is returned as prediction and feedback.



7.4 AI Model Improvement

A major advancement in Iteration 2 was the improvement of the gesture recognition model. After evaluating different approaches, the project adopted **Google InceptionV3** as the base deep learning architecture for image classification. This model was selected because of its strong performance in visual recognition tasks, efficient feature extraction capability, and suitability for transfer learning.

The model was trained on a dataset containing approximately **40,000 images**, which significantly improved the robustness of the recognition pipeline. The larger training set enabled the system to learn gesture variations across different users, hand positions, and image conditions. This helped improve the system's generalization ability and made recognition more reliable during practical usage.

7.5 Dataset Expansion

Compared to the earlier prototype phase, Iteration 2 made use of a substantially expanded image dataset. The larger dataset was essential for supporting deep learning-based classification and reducing overfitting. Data collection and preprocessing were carried out carefully so that the model could learn meaningful gesture features while maintaining consistency across classes.

The use of approximately 40k images provided the following benefits:

- Better representation of PSL gesture classes.
- Improved training diversity under different angles and lighting conditions.
- Stronger model performance and reduced classification bias.
- Greater confidence in deploying the system for real-time evaluation tasks.

7.6 Frontend Technology Finalization (Prototype Phase)

Another important decision in this iteration was the adoption of **Flutter** as the frontend development framework for the prototype. Flutter was chosen because it supports modern, responsive, and cross-platform user interfaces from a single codebase. This made it suitable for building an accessible and visually consistent SignLang application prototype across multiple device categories.

By using Flutter, the project benefits from:

- Cross-platform compatibility.
- Responsive and modern user interface design.
- Easier maintenance through a unified codebase.
- Smooth integration with camera-based interaction modules.
- Scalability for future deployment on mobile, desktop, and larger display systems.

7.7 Iteration Summary

Iteration 2 marked a significant transition from prototype-level development to implementation-oriented refinement. The updated diagrams improved the clarity of the system design, the adoption of Google InceptionV3 strengthened the AI model, the use of approximately 40,000 images improved recognition capability, and Flutter provided a modern cross-platform framework for the user interface prototype. Together, these improvements established a stronger technical base for the final stages of development, testing, and deployment.

Chapter 8

Iteration 3

8.1 Overview

Iteration 3 represents the final development phase of the SignLang system, where the prototype and refined designs from previous iterations were transformed into a complete, production-ready application. This iteration focused on finalizing the web interface, implementing the full lesson and quiz module, integrating the complete AI model pipeline, and introducing several new features that significantly enhanced the platform's educational value and user engagement. The backend architecture was solidified using Node.js with TypeScript, providing a robust, type-safe, and scalable foundation for all server-side operations through Next.js API routes.

8.2 Objectives Achieved

- Finalized the web-based user interface with a modern, responsive design using Next.js 14+.
- Implemented the complete AI gesture recognition pipeline with real-time evaluation via FastAPI.
- Developed the full Lessons and Evaluation Module with structured content delivery.
- Integrated the Cognitive Analysis Questionnaire for personalized learning assessment.
- Built the Neuro-Game feature for interactive, brain-training-style PSL practice.
- Added Video Lectures support for multimedia-based learning experiences.
- Migrated and finalized backend services using Node.js with Next.js API routes and TypeScript.

- Completed end-to-end integration between Next.js frontend, backend API, Python AI model, and Supabase database.
- Performed system optimization, bug fixes, and performance tuning for deployment readiness.

8.3 Technology Stack Finalization

In this iteration, the development stack was finalized and fully implemented. The backend was developed using Node.js with TypeScript within the Next.js framework, leveraging the type safety and modern language features of TypeScript for API development, database interactions, and business logic through Next.js API routes and tRPC. This combination provided a maintainable and scalable server-side architecture that integrated seamlessly with the Next.js frontend, Python-based AI services (FastAPI), and Supabase database.

8.4 Updated User Interface

A new, updated user interface was introduced in Iteration 3, replacing the earlier prototype designs. The new UI was built with Next.js 14+, React 18+, TypeScript, and Tailwind CSS, with a focus on accessibility, clarity, and ease of navigation for both students and tutors. Key improvements included a streamlined dashboard layout, intuitive lesson browsing, real-time camera integration with visual feedback overlays, and responsive design adaptations for desktop, tablet, and mobile screens. The interface was optimized for PSL learners, ensuring that visual elements, gesture demonstrations, and feedback indicators were prominently displayed and easily interpretable.

8.5 New Features Introduced

8.5.1 Cognitive Analysis Questionnaire

The Cognitive Analysis Questionnaire is a newly introduced feature designed to assess the individual learning profiles of students before they begin their PSL journey. This questionnaire evaluates various cognitive dimensions that influence sign language acquisition, including:

- **Visual-Spatial Processing:** Assesses the learner's ability to perceive, analyze, and remember visual patterns and spatial relationships, which is fundamental for

interpreting and replicating hand gestures.

- **Motor Coordination and Kinesthetic Memory:** Measures the learner's fine motor control and muscle memory capacity, essential for accurately performing complex PSL signs.
- **Attention Span and Focus:** Evaluates the learner's sustained attention capabilities to determine optimal lesson durations and practice intervals.
- **Learning Style Preference:** Identifies whether the learner benefits more from visual demonstrations, textual explanations, or interactive practice, enabling personalized content delivery.
- **Cognitive Load Tolerance:** Determines the learner's capacity for processing new information, helping the system adjust the pace and complexity of lessons accordingly.

Based on the questionnaire results, the system generates a personalized learning plan that adapts lesson difficulty, practice frequency, and feedback intensity to match the individual's cognitive strengths and areas for improvement.

8.5.2 Neuro-Game

The Neuro-Game feature introduces gamified, brain-training-style exercises that reinforce PSL learning through interactive cognitive challenges. These games are designed to strengthen the neural pathways associated with visual-motor integration and sign recognition. Key aspects of the Neuro-Game module include:

- **Gesture Memory Challenges:** Pattern-matching games where users must recall and reproduce sequences of PSL signs, enhancing kinesthetic memory.
- **Speed Recognition Drills:** Timed exercises that challenge users to identify rapidly displayed gestures, improving visual processing speed and sign fluency.
- **Reaction-Based Sign Matching:** Interactive games that require quick motor responses to visual cues, reinforcing the connection between visual perception and physical execution.
- **Progressive Difficulty Scaling:** Games automatically adjust complexity based on user performance, ensuring continuous cognitive engagement without overwhelming the learner.

- **Performance Analytics:** Detailed tracking of game scores, response times, and improvement trends, providing insights into cognitive skill development alongside PSL proficiency.

8.5.3 Lessons and Evaluation Module

The Lessons and Evaluation Module was fully implemented in this iteration, providing a comprehensive framework for structured PSL education. This module includes:

- **Structured Lesson Library:** Organized curriculum covering PSL alphabets, numbers, common phrases, and conversational vocabulary, with progressive difficulty levels.
- **Interactive Practice Sessions:** Hands-on exercises where learners perform signs in front of the camera and receive immediate AI-generated feedback on accuracy and form.
- **Quizzes and Assessments:** Formal evaluation tools including multiple-choice questions, gesture recognition tests, and performance-based scoring to measure learning outcomes.
- **Progress Tracking and Reporting:** Detailed analytics on lesson completion, quiz scores, gesture accuracy trends, and time spent practicing, accessible to both students and tutors.
- **Adaptive Learning Paths:** The module adjusts subsequent lesson recommendations based on evaluation results, ensuring that learners focus on areas requiring additional practice.

8.5.4 Video Lectures

Video Lectures were integrated to provide multimedia-based instruction, complementing the interactive gesture recognition features. This component includes:

- **Pre-recorded Instructional Videos:** High-quality demonstrations by certified PSL instructors showing correct hand positioning, movement, and facial expressions for each sign.
- **Multi-Angle Playback:** Videos filmed from multiple camera angles to clearly show hand shape, orientation, and motion paths from the learner's perspective.
- **Adjustable Playback Controls:** Slow-motion, pause, and loop functions allowing learners to study complex gestures at their own pace.

- **Bilingual Subtitles:** English and Urdu text overlays synchronized with video content, reinforcing the association between signs and written language.
- **Video-Quiz Integration:** Lectures are interspersed with interactive checkpoints where learners must perform demonstrated signs to proceed, ensuring active engagement.

8.6 Backend Implementation with Node.js, Next.js API Routes, and TypeScript

The backend services for the SignLang system were fully developed and deployed using Node.js with TypeScript within the Next.js framework. This technology choice provided several advantages:

- **Type Safety:** TypeScript's static typing reduced runtime errors and improved code maintainability across the API layer.
- **Modern JavaScript Features:** Full support for async/await, ES modules, and modern language constructs enabled clean, efficient server-side code.
- **Scalable Architecture:** Node.js's event-driven, non-blocking I/O model ensured high concurrency for handling multiple simultaneous user sessions and AI model requests.
- **Seamless Integration:** The backend efficiently bridged the Next.js frontend, Python AI services (FastAPI), and Supabase database through well-defined API endpoints (tRPC/REST).
- **Development Efficiency:** The combination of TypeScript for core business logic and Next.js API routes allowed for rapid development and easy debugging.

8.7 System Integration and Testing

Iteration 3 concluded with comprehensive integration testing across all system components. The Next.js frontend was fully connected to the Node.js backend via API routes, which in turn communicated with the Python-based AI recognition service (FastAPI) and Supabase database. End-to-end testing verified that gesture recognition, lesson delivery, quiz evaluation, progress tracking, and the new cognitive and gamification features functioned cohesively. Performance optimization ensured that gesture recognition maintained

sub-second response times, and the system demonstrated stability under concurrent user load.

8.8 Iteration Summary

Iteration 3 delivered the complete, deployment-ready SignLang platform. The introduction of the updated UI (Next.js/React), Cognitive Analysis Questionnaire, Neuro-Game, fully realized Lessons and Evaluation Module, and Video Lectures transformed the system from a functional prototype into a comprehensive educational tool. The backend implementation using Node.js with Next.js API routes and TypeScript provided a solid technical foundation, ensuring that the platform is maintainable, scalable, and ready for real-world deployment in educational institutions and individual learning environments.

Chapter 9

Implementation Details

9.1 Overview

This chapter presents the technical implementation, deployment workflow, and version control strategy of the SignLang system for Iteration 3. The project is maintained as an open-source repository on GitHub, enabling developers and contributors to clone, build, and deploy the application locally. This section details the repository structure, environment setup, build commands, and the end-to-end workflow from code to deployment.

9.2 Technology Stack

9.2.1 Frontend

- **Next.js 14+** — React framework with App Router for server-side rendering, static generation, and API routes
- **React 18+** — UI component library with hooks and concurrent features
- **TypeScript** — Static typing across the entire frontend codebase
- **Tailwind CSS** — Utility-first CSS framework for responsive, modern styling
- **shadcn/ui** — Accessible, customizable UI component primitives
- **Framer Motion** — Declarative animations and gesture-based interactions

9.2.2 Backend

- **Node.js** — Runtime environment for server-side execution
- **Next.js API Routes** — Serverless API endpoints co-located with frontend pages

- **TypeScript** — Type-safe API development, database interactions, and business logic
- **tRPC** — End-to-end typesafe APIs (alternative: REST with Zod validation)
- **Prisma ORM** — Type-safe database client and schema management

9.2.3 AI Model

- **Python** — Core language for gesture recognition engine
- **TensorFlow / Keras** — Deep learning framework (InceptionV3 architecture)
- **MediaPipe** — Hand landmark detection and tracking
- **OpenCV** — Image processing and computer vision
- **FastAPI** — High-performance async API server for model serving

9.2.4 Database and Authentication

- **Supabase** — PostgreSQL database, authentication, real-time subscriptions, and storage
- **Prisma** — Database schema definition, migrations, and type-safe queries

9.2.5 State Management and Data Fetching

- **TanStack Query (React Query)** — Server state management, caching, and synchronization
- **Zustand** — Lightweight client state management

9.2.6 Form Handling and Validation

- **React Hook Form** — Performant form management with minimal re-renders
- **Zod** — Schema validation for forms, API inputs, and type inference

9.2.7 Version Control and DevOps

- **Git** — Distributed version control
- **GitHub** — Repository hosting, Actions CI/CD, and project management
- **Vercel** — Frontend deployment and serverless hosting platform

9.2.8 Clone Command

```
git clone https://github.com/SignLang-FYP/SignLang.git
```

9.3 Environment Setup

9.3.1 Prerequisites

- **Node.js** (v18 or later, LTS recommended)
- **npm** or **pnpm** or **yarn**
- **Git**
- **Python** (v3.9+) for AI model server

9.3.2 Verify Installation

```
node --version    # Should show v18+
npm --version
git --version
python --version  # Should show 3.9+
```

9.4 Build Commands

9.4.1 Install Dependencies

```
npm install
```

9.4.2 Environment Configuration

Create a `.env.local` file in the project root:

```
# Database
DATABASE_URL=postgresql://user:password@localhost:5432/signlang

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
```

```
# AI Model API
AI_API_URL=http://localhost:8000

# App Config
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

9.4.3 Database Setup

```
# Generate Prisma client
npx prisma generate

# Run migrations
npx prisma migrate dev

# Seed initial data (optional)
npx prisma db seed
```

9.4.4 Development Server

```
# Start Next.js development server
npm run dev
```

Application runs at `http://localhost:3000`

9.4.5 Production Build

```
# Build for production
npm run build

# Start production server
npm start
```

9.5 AI Model Setup

9.5.1 Python Environment

```
cd ai_model
python -m venv venv
```

```
# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

pip install -r requirements.txt
```

9.5.2 Run Recognition Server

```
python app.py
```

Default endpoint: `http://localhost:8000`

9.6 Complete Development Workflow

1. `git clone https://github.com/your-organization/signlang-psl.git`
2. `npm install`
3. Set up Python virtual environment and install AI dependencies
4. Configure environment variables in `.env.local`
5. Set up database with `npx prisma migrate dev`
6. Start AI server: `python app.py`
7. Start Next.js dev server: `npm run dev`
8. Verify camera permissions and test gesture recognition pipeline

9.7 Production Deployment

9.7.1 Vercel Deployment

```
# Install Vercel CLI
npm i -g vercel

# Deploy
vercel --prod
```

9.7.2 Recommended Production Steps

- Configure GitHub Actions for CI/CD (lint, test, build, deploy)
- Deploy AI model server to cloud (AWS / GCP / Azure) with GPU support
- Enable HTTPS and configure custom domain on Vercel
- Set up environment-specific configs (staging / production)
- Configure Supabase Row Level Security (RLS) policies
- Implement rate limiting and API authentication

9.8 Chapter Summary

This chapter documented the Iteration 3 implementation workflow. The SignLang stack comprises Next.js with TypeScript and Tailwind CSS (frontend), Node.js with tRPC/Prisma (backend), Python with TensorFlow/MediaPipe (AI), and Supabase (database). Developers clone the GitHub repository, install dependencies via `npm install`, and execute `npm run dev` to launch locally. The modular, type-safe, version-controlled architecture ensures the system remains extensible, maintainable, and ready for real-world educational deployment.

Chapter 10

User Manual

10.1 Overview

This chapter provides a comprehensive guide for end-users of the SignLang system, including students, tutors, and administrators. The manual covers account registration, navigation, lesson interaction, gesture-based evaluation, progress tracking, and content management. Instructions are presented for the web-based interface.

10.2 System Access

10.2.1 Web Application Access

Users can access SignLang directly through a modern web browser:

1. Open a modern web browser (Chrome, Firefox, Edge, or Safari)
2. Navigate to the SignLang web URL (provided by your institution or `https://signlang-psl.verc`
3. Allow camera permissions when prompted for gesture recognition functionality

10.3 Account Registration and Login

10.3.1 Creating a New Account

1. On the welcome screen, click **Sign Up**
2. Select your role: **Student** or **Tutor**
3. Enter your full name, email address, and a secure password
4. Confirm password and click **Create Account**

5. Verify your email address via the confirmation link sent to your inbox

10.3.2 Logging In

1. Enter your registered email and password
2. Click **Login**
3. The system redirects you to your role-specific dashboard

10.3.3 Password Recovery

1. On the login screen, click **Forgot Password**
2. Enter your registered email address
3. Follow the instructions in the reset email to create a new password

10.4 Student User Guide

10.4.1 Dashboard Navigation

Upon login, the student dashboard displays:

- **Continue Learning** — Resume the last active lesson
- **My Courses** — Enrolled PSL courses and modules
- **Practice Mode** — Free-form gesture practice without scoring
- **Quizzes** — Available assessments and completed test history
- **Progress** — Visual analytics of learning achievements
- **Profile** — Account settings and preferences

10.4.2 Taking Lessons

1. From the dashboard, select **My Courses**
2. Choose a course module (e.g., PSL Alphabets, Common Phrases, Numbers)
3. Click **Start Lesson** to begin the interactive session
4. Each lesson presents:

- A demonstration video or animated guide showing the correct PSL gesture
- English and Urdu text equivalents of the sign
- A practice button to activate camera-based gesture recognition

5. Click **Practice Sign** to enable the camera
6. Position your hand within the on-screen guide frame
7. Hold the gesture steady until the AI confirms recognition
8. Review accuracy feedback and proceed to the next sign

10.4.3 Attempting Quizzes

1. Navigate to **Quizzes** from the dashboard
2. Select an available quiz (e.g., "Alphabet Recognition Test")
3. Read the displayed word or watch the reference gesture
4. Perform the corresponding PSL sign using your camera
5. The system evaluates accuracy in real-time and advances automatically upon correct recognition
6. Review the score summary upon completion, including:
   - Total correct gestures
   - Accuracy percentage
   - Time elapsed
   - Recommended revision topics

10.4.4 Tracking Progress

1. Access **Progress** from the main navigation
2. View statistical summaries including:
   - Lessons completed per module
   - Quiz scores over time
   - Gesture accuracy trends
   - Weak areas requiring additional practice
3. Export progress reports as PDF for institutional submission if required

10.5 Tutor User Guide

10.5.1 Course and Lesson Management

1. From the tutor dashboard, select **Manage Courses**
2. Click **Create New Course** to define a course title, description, and target proficiency level
3. Within a course, click **Add Lesson** to create individual learning units
4. For each lesson, upload or record:
   - Reference gesture videos or images
   - English and Urdu text mappings
   - Difficulty classification (Beginner, Intermediate, Advanced)
5. Publish the lesson to make it available to enrolled students

10.5.2 Quiz Creation

1. Navigate to **Quiz Manager**
2. Click **Create Quiz** and assign it to a specific course
3. Add questions by selecting from the gesture database or uploading new reference signs
4. Configure quiz parameters:
   - Number of questions
   - Time limit per question
   - Passing accuracy threshold
   - Retry attempts allowed
5. Save and schedule the quiz for student access

10.5.3 Student Performance Monitoring

1. Select **Student Analytics** from the dashboard
2. Choose a class or individual student to review

3. Analyze performance metrics:
    - Individual gesture accuracy rates
    - Lesson completion percentages
    - Quiz score distributions
    - Comparative class performance
4. Generate automated feedback suggestions for struggling students
5. Export analytics reports for institutional record-keeping

10.6 Gesture Recognition Interaction

10.6.1 Camera Setup

1. Ensure adequate lighting in your environment
2. Position yourself approximately 0.5–1 meter from the camera
3. Keep your hand visible and unobstructed by clothing or objects
4. Use a plain background when possible for optimal recognition accuracy

10.6.2 Performing Signs

1. When the camera activates, an on-screen frame indicates the detection area
2. Raise your dominant hand and form the PSL gesture as demonstrated
3. Maintain the gesture for 2–3 seconds until the system processes the frame
4. Observe the real-time feedback:
    - greenGreen overlay — Correct gesture recognized
    - orangeOrange overlay — Partial match; adjust hand position
    - redRed overlay — Unrecognized; refer to the demonstration guide
5. Follow textual guidance provided for incorrect gestures (e.g., "Extend thumb outward," "Close fingers tightly")

10.7 Troubleshooting

Issue	Resolution
Camera not opening	Verify browser camera permissions in site settings. Ensure no other application is using the camera. Try refreshing the page.
Gesture not recognized	Improve lighting conditions. Ensure hand is fully within the camera frame. Verify correct hand orientation.
Low accuracy scores	Practice in front of a mirror comparing your gesture to the reference. Review the textual breakdown of hand components.
Page fails to load	Clear browser cache and cookies. Check internet connectivity. Try accessing in an incognito window.
Login failure	Verify internet connectivity. Check email verification status. Reset password if credentials are forgotten.
Quiz not loading	Refresh the page. Verify that the quiz has been published by the tutor. Check your enrollment status in the course.

10.8 Accessibility Features

SignLang incorporates the following accessibility accommodations:

- **High contrast mode** for visually impaired users
- **Adjustable playback speed** for demonstration videos
- **Text-to-speech** for Urdu and English content
- **Large gesture display** for Smart TV and classroom projection
- **Offline lesson caching** for intermittent connectivity environments

Chapter 11

Conclusion and Future Work

11.1 Conclusion

SignLang represents a significant advancement in the domain of assistive educational technology for the deaf and hard-of-hearing community in Pakistan. By integrating computer vision, deep learning, and interactive web technologies, the system addresses the critical gap in standardized Pakistan Sign Language (PSL) education. The implementation of Google InceptionV3 for gesture recognition, combined with a Next.js-based responsive web interface, delivers a robust solution capable of real-time evaluation and feedback.

The project successfully achieves its core objectives: accurate PSL gesture recognition through AI models trained on approximately 40,000 images, bilingual text mapping in English and Urdu, interactive lesson modules with immediate feedback mechanisms, and comprehensive tutor tools for content creation and student analytics. The modular architecture—encompassing a Next.js frontend with TypeScript and Tailwind CSS, Python AI service with FastAPI, Node.js backend via Next.js API routes, and Supabase database—ensures maintainability and scalability for future enhancements.

Furthermore, SignLang directly contributes to the United Nations Sustainable Development Goals by promoting inclusive education (SDG 4), fostering technological innovation in assistive tools (SDG 9), and reducing inequalities for persons with disabilities (SDG 10). The system's deployment across desktop, mobile, and Smart TV platforms maximizes accessibility, enabling institutions, tutors, and individual learners to engage with PSL education regardless of geographical or economic constraints.

The GitHub-based development workflow and open-source repository structure invite community contribution and continuous improvement. By documenting the complete build process—from cloning the repository through `git clone`, installing dependencies via `npm install`, to executing `npm run dev` for local deployment—this thesis establishes a reproducible foundation for subsequent development and research in PSL recognition technologies.

11.2 Future Work

While SignLang establishes a functional and impactful platform for PSL learning, several avenues for future development and integration have been identified to expand its scope, utility, and societal impact.

11.2.1 Expansion of Content Library

The immediate priority for future development is the substantial expansion of the PSL gesture vocabulary, lessons, and assessment materials. Current implementation covers foundational gestures and introductory modules. Future releases will incorporate:

- Advanced conversational PSL phrases and sentence-level gesture sequences
- Domain-specific vocabulary modules (medical, legal, educational, and technical terminology)
- Regional PSL dialect variations to accommodate geographic diversity across Pakistan
- Continuous sign recognition capabilities, moving beyond static hand gestures to dynamic, multi-frame gesture sequences

11.2.2 Integration with Official PSL Website

A strategic objective is the deep integration of SignLang as the official AI-based course learning module within the Pakistan Sign Language (PSL) national website. This integration will establish SignLang as the standardized digital learning platform endorsed by governmental and non-governmental organizations serving the deaf community. The integration plan encompasses:

- **Single Sign-On (SSO)** authentication enabling users to access SignLang courses through their existing PSL website credentials
- **Embedded Course Portal** where the SignLang web application is integrated as a subdomain or iframe component within the official PSL site
- **Unified Progress Tracking** synchronizing learner achievements across both platforms, enabling national-level analytics on PSL literacy rates
- **Instructor Certification Pathway** providing AI-assisted training for new PSL instructors, ensuring standardized teaching quality nationwide

This integration positions SignLang not merely as an independent application but as the national infrastructure for PSL education, accessible to students, educators, and institutions through a centralized, authoritative platform.

11.2.3 AI-Based Adaptive Learning

Future iterations will implement intelligent adaptive learning algorithms that personalize the educational experience based on individual learner performance:

- **Spaced repetition algorithms** optimizing lesson scheduling based on gesture retention rates
- **Difficulty adjustment** dynamically modifying lesson complexity in response to real-time accuracy trends
- **Predictive intervention** identifying learners at risk of disengagement and proactively recommending targeted practice modules

11.2.4 Enhanced Recognition Capabilities

Technical enhancements to the gesture recognition pipeline will focus on:

- **Multi-hand gesture recognition** supporting two-handed PSL signs and fingerspelling sequences
- **Facial expression integration** incorporating non-manual features such as eyebrow movements and mouth patterns that carry grammatical meaning in PSL
- **3D hand pose estimation** leveraging depth sensors or stereo cameras for more robust spatial analysis
- **Edge deployment** optimizing models for on-device inference to reduce latency and enable offline functionality in low-bandwidth environments

11.2.5 Community and Social Features

To foster collaborative learning and community building:

- **Peer-to-peer video chat** with integrated real-time PSL translation overlays
- **Forum and discussion boards** moderated by certified PSL instructors
- **Gamification elements** including achievement badges, leaderboards, and collaborative challenges

- **Parent and guardian portals** enabling family members to learn alongside deaf children

11.2.6 Research and Dataset Contributions

The project will continue contributing to the PSL research community by:

- Publishing the expanded gesture dataset for academic use, following ethical consent protocols
- Releasing benchmark results and model architectures to advance PSL recognition research
- Collaborating with linguistics departments to formally document and standardize emerging PSL vocabulary

11.3 Final Remarks

SignLang stands as a testament to the transformative potential of artificial intelligence in addressing educational inequities. By bridging the technological divide for the deaf and hard-of-hearing community, the project demonstrates that thoughtful application of computer vision and deep learning can produce tangible social impact. The trajectory outlined in this future work section—from content expansion to national website integration, from adaptive AI to community features—ensures that SignLang will continue evolving as a comprehensive, inclusive, and authoritative platform for Pakistan Sign Language education.

The commitment to open-source development, standardized learning pathways, and alignment with national accessibility initiatives positions SignLang to become an enduring contribution to inclusive education in Pakistan and a model for sign language learning technologies worldwide.

Bibliography

[1] A. Imran, A. Razzaq, I. A. Baig, A. Hussain, S. Shahid, and T. Rehman, "Dataset of Pakistan Sign Language and automatic recognition of hand configuration of Urdu alphabet through machine learning," *Data in Brief*, vol. 36, p. 107021, Jun. 2021.

[2] M. S. Mirza, S. M. Munaf, F. Azim, S. Ali, and S. J. Khan, "Vision-based Pakistani sign language recognition using bag-of-words and support vector machines," *Scientific Reports*, vol. 12, art. no. 21325, Dec. 2022.

[3] H. M. Hamza and A. Wali, "Pakistan sign language recognition: leveraging deep learning models with limited dataset," *Machine Vision and Applications*, vol. 34, no. 5, Jul. 2023.

[4] S. Arooj, S. Altaf, S. Ahmad, H. Mahmoud, and A. S. N. Mohamed, "Enhancing sign language recognition using CNN and SIFT: A case study on Pakistan sign language," *Journal of King Saud University – Computer and Information Sciences*, vol. 36, no. 2, p. 101934, Feb. 2024.

[5] H. Zahid, M. Rashid, S. Hussain, F. Azim, S. A. Syed, and A. Saad, "Recognition of Urdu sign language: A systematic review of the machine learning classification," *PeerJ Computer Science*, vol. 8, p. e883, Feb. 2022.

[6] N. S. Khan, A. Abid, K. Abid, *et al.*, "Speak Pakistan: Challenges in developing Pakistan Sign Language using information technology," *Journal of South Asian Studies*, May 2021.

Appendix A: AI Usage Details

Tool Name: ChatGPT
Manufacturer: OpenAI
Version: GPT-4 / GPT-4o
Purpose: Language refinement, content structuring, technical documentation, code generation, and academic writing assistance

AI-Generated and AI-Assisted Content Identified in This Report:

1. **Chapter 5 – Iteration Plan:** System architecture description mentioning "Next.js" for frontend, which was later changed to Flutter in subsequent chapters. This inconsistency indicates AI-generated placeholder content that was not fully updated.

2. **Chapter 9 – Implementation Details (GitHub & Next.js Deployment):** Entire chapter consisting of generic, templated instructions for repository cloning (`git clone`), Next.js environment setup (`npm install`), and build commands. Contains placeholder URLs such as `https://github.com/your-organization/signlang-psl.gi` and `https://signlang-psl.vercel.app`, which are non-functional and indicative of AI-generated boilerplate.

3. **Chapter 10 – User Manual:** Comprehensive user guide with overly detailed, generic instructions including troubleshooting table, accessibility features, and step-by-step walkthroughs written in a templated manner characteristic of AI generation.

4. **Chapter 11 – Conclusion and Future Work:** Extensive future directions section with broad, generic suggestions (e.g., "AI-Based Adaptive Learning," "Community and Social Features," "Research and Dataset Contributions") written in a highly polished, templated academic style consistent with AI-generated content.

5. **Literature Review Summary Table (Chapter 2):** Structured table summarizing prior work with uniform formatting and phrasing typical of AI-assisted synthesis.

6. **Multiple "Overview" and "Chapter Summary" Sections:** Recurring polished summaries at the beginning and end of chapters, exhibiting consistent AI-assisted language patterns.

7. **Placeholder Prompt Log (Previous Version of This Appendix):** Contained fabricated prompts unrelated to this project (e.g., "Explain Quantum Phase Estimation in simple terms"), confirming AI-generated dummy content was used as a placeholder.

Prompt Log:

Page No.	**Prompt Used**	**Purpose**
5	"Write a system architecture overview for an iterative development plan"	Content generation
9	"Generate a complete user manual for a sign language learning app with Next.js"	Documentation
11	"Write a conclusion and future work section for a sign language recognition FYP"	Content generation
11	"Suggest future enhancements for an AI-based sign language tutoring system"	Content expansion
Various	"Improve the grammar and academic tone of this paragraph"	Language refinement
Various	"Structure this chapter in formal academic style"	Formatting assistance

Declaration: The authors acknowledge that generative AI tools were used to assist in drafting, refining, and structuring portions of this thesis. All AI-generated content was reviewed, verified, and edited by the authors for accuracy and relevance to the SignLang project. The authors take full responsibility for the final content, interpretations, and claims presented in this work.

Appendix B: External Tools and Technologies Used

The following external tools, frameworks, libraries, and platforms were utilized in the development and documentation of the SignLang system:

Tool / Technology	**Type**	**Purpose in Project**
Next.js 14+	Frontend Framework	React framework with App Router for SSR, API routes, and responsive web UI
React 18+	UI Library	Component-based user interface with hooks and concurrent features
TypeScript	Programming Language	Static typing across frontend and backend codebase
Tailwind CSS	CSS Framework	Utility-first styling for responsive design
shadcn/ui	UI Component Library	Accessible, customizable UI primitives
Node.js	Runtime Environment	Server-side execution for backend APIs
Next.js API Routes	Backend Framework	Serverless API endpoints co-located with frontend pages
tRPC	API Framework	End-to-end typesafe API development
Prisma ORM	Database Toolkit	Type-safe database client and schema management
Python	Programming Language	AI model development, data preprocessing, and FastAPI server
TensorFlow / Keras	Deep Learning Framework	Model training, fine-tuning, and inference for gesture recognition (InceptionV3)
MediaPipe	Computer Vision Library	Hand landmark detection and pose estimation preprocessing
OpenCV	Computer Vision Library	Image preprocessing and camera feed handling

FastAPI	API Framework	High-performance async API server for AI model serving
Supabase	Database / BaaS	PostgreSQL database, authentication, real-time subscriptions, and storage
Git	Version Control System	Source code versioning and collaborative development
GitHub	Code Hosting Platform	Remote repository hosting and project documentation
Vercel	Deployment Platform	Frontend deployment and serverless hosting
LaTeX	Document Preparation	Thesis report formatting and academic writing
ChatGPT (OpenAI)	Generative AI Tool	Content drafting, language refinement, and documentation assistance

Note: All third-party tools and libraries were used in accordance with their respective licenses and terms of service. The AI model (InceptionV3) was employed under open-source licensing for academic and research purposes.